

# C# Programming Language

---

Firouzeh Raygan

Professor : Saeed Talatian Azad





# History of C#

- Languages before C#
  - C
  - C++
  - JAVA
- Microsoft and JAVA
- Microsoft vs Sun Microsystems
- Andres Hejlsberg
- C# Project
- Naming



# History of C#

- Languages before C#
- Microsoft and JAVA : MSJVM
- Microsoft vs Sun Microsystems
- Andres Hejlsberg
- C# Project
- Naming



# History of C#

- Languages before C#
- Microsoft and JAVA
- Microsoft vs Sun Microsystems
- Andres Hejlsberg
- C# Project
- Naming



Borland:

- Turbo Pascal
- Delphi

Microsoft:

- C#
- .NET



# History of C#

- Languages before C#
  - Microsoft and JAVA
  - Microsoft vs Sun Microsystems
  - Andres Hejlsberg
  - C# Project
  - Naming
- Simplicity
  - Type Safety & Memory Safety
  - Object-Oriented Programming
  - .NET Integration
  - Rapid Application Development
  - Power with Productivity
  - Scalability



# History of C#

- Languages before C#
- Microsoft and JAVA
- Microsoft vs Sun Microsystems
- Andres Hejlsberg
- C# Project
- Naming



Microsoft

the late '90s



# .NET PLATFORM

| Layer          | Responsibility  |
|----------------|-----------------|
| Language Layer | VB, F# , C#     |
| Runtime Layer  | CLR و GC و JIT  |
| Library Layer  | BCL و Framework |

- CLR {
  - Memory management
  - Garbage collection
  - Execution

- BCL {
  - File handling
  - Networking
  - Strings
  - Database
  - Threading
  - Collections

- CTS(Common Type System)
- CLS(Common Language Specification)
  - type sharing and defining the rules

- COMPILERS
- GC(Garbage Collector )

- ✓ .NET FRAMEWORK
- ✓ .NET CORE
- ✓ .NET



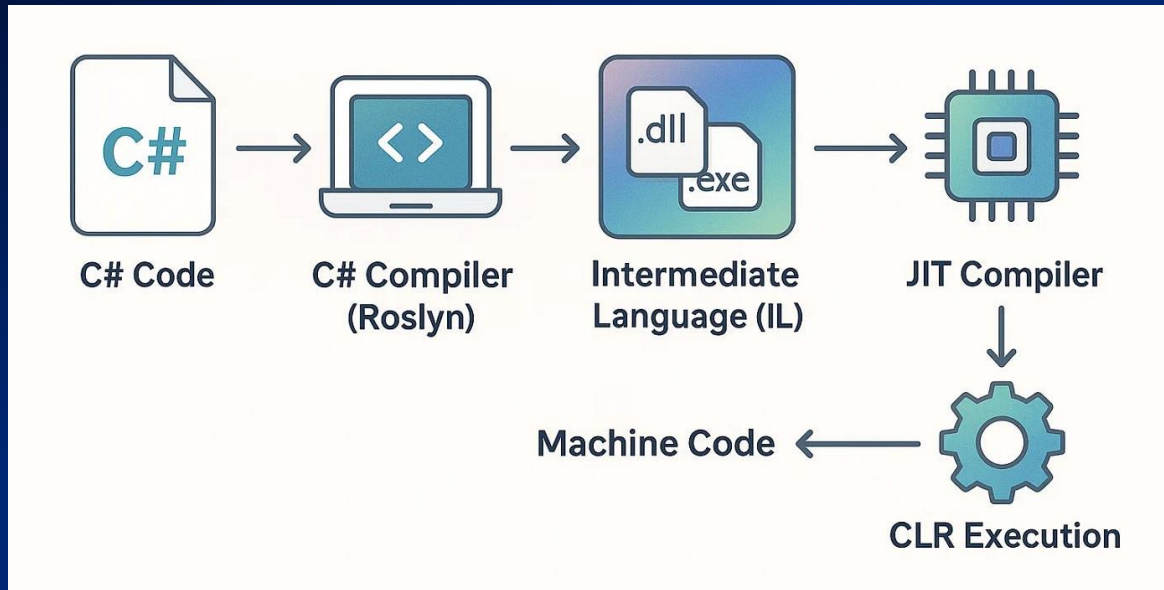
# FRAMEWORKS

- **ASP.NET** Website → ASP .NET CORE
- **WIN FORMS** Windows Desktop Applications
- **MAUI.NET** Apps for Android , iOS, Windows ,macoS with same basecode
- **BLAZER** Web FrontEnd , use C# instead of javaScript
- **ENTITY.NET** Database
- **ML.NET** AI, Machine Learning
- **UNITY** Games





# COMPILER



- ✓ JIT(Just In Time)
- ✓ AOT(Ahead Of Time)



# GENERAL STRUCTURE

```
using System;    // وارد کردن فضای نام

namespace MyFirstApp
{
    0 references
    class Program
    {
        0 references
        static void Main(string[] args)    // نقطه ورود برنامه
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```



# C# BASIC RULES

- Case Sensitivity      Name  $\neq$  name

- Semicolon

- Block

- Comments

```
// Single-line comment

/*
    Multi-line comment
*/

/// XML Documentation Comment
```

```
using System;    // وارد کردن فضای نام

namespace MyFirstApp
{
    0 references
    class Program
    {
        0 references
        static void Main(string[] args)    // نقطه ورود برنامه
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```



# PRIMITIVE DATA TYPES

- Value Type
- Reference Type

| Type    | Category       | Size     | Range / Value                                         | Description                 |
|---------|----------------|----------|-------------------------------------------------------|-----------------------------|
| int     | Value Type     | 4 Bytes  | -2B to 2B                                             | Integer numbers             |
| float   | Value Type     | 4 Bytes  | $\pm 1.5 \times 10^{-45}$ to $\pm 3.4 \times 10^{38}$ | Single precision decimal    |
| double  | Value Type     | 8 Bytes  | Very large decimal range                              | Double precision decimal    |
| decimal | Value Type     | 16 Bytes | 28-29 digits precision                                | Financial calculations      |
| char    | Value Type     | 2 Bytes  | Unicode character                                     | Single character            |
| bool    | Value Type     | 1 Byte   | true / false                                          | Logical values              |
| byte    | Value Type     | 1 Byte   | 0 to 255                                              | Small positive integers     |
| long    | Value Type     | 8 Bytes  | Very large integers                                   | Large whole numbers         |
| string  | Reference Type | Variable | Text values                                           | Sequence of characters      |
| object  | Reference Type | Variable | Any type                                              | Base type of all data types |



- **Var** determined at compile time and it cannot be null.

- **Consting**

```
const double pi = 3.14159;  
const int weekdays = 7;
```

- **Operators**

```
// Arithmetic Operators
```

```
int sum = 5 + 3;           // 8  
int subtraction = 10 - 4;  // 6  
int multiply = 6 * 7;      // 42  
int divide = 15 / 3;       // 5  
int remains = 17 % 5;      // 2
```

```
// Logical Operators
```

```
bool and = (true && false); // false  
bool or = (true || false);  // true  
bool not = !true;           // false
```

```
// Comparison Operators
```

```
bool equal = (5 == 5);      // true  
bool unequal = (5 != 3);    // true  
bool greaterThan = (10 > 5); // true  
bool lessThan = (3 < 7);    // true  
bool greaterThanOrEqual = (8 >= 5); // true  
bool lessThanOrEqual = (7 <= 7); // true
```

```
//String Operators
```

```
string firstName = "Ali";  
string lastName = "Rezaei";  
string fullName = firstName + " " + lastName;
```

```
string hello = "Hello" + " " + "World";  
string ageText = "My age is " + 22 + " years old";
```

```
// Assignment Operators
```

```
int x = 10;  
x += 5;    // 15  
x -= 3;    // 12  
x *= 2;    // 24  
x /= 4;    // 6
```



- **Var** determined at compile time and it cannot be null.

- **Consting**

```
const double pi = 3.14159;  
const int weekdays = 7;
```

- **Operators**
- **Static Type checking**
- **Eager evaluation**

# CONTROL STATEMENTS



## Conditionals

- If-else
- Switch

```
static string GetGradeOld(int score)
{
    switch (score)
    {
        case int s when s >= 90:
            return "A";
        case int s when s >= 80:
            return "B";
        case int s when s >= 70:
            return "C";
        default:
            return "F";
    }
}
```

Old V

```
static string GetGradeNew(int score) => score switch
{
    >= 90 => "A",
    >= 80 => "B",
    >= 70 => "C",
    _ => "F"
};
```

new V

# CONTROL STATEMENTS



## Conditionals

- If-else
- Switch

## Loops

- For
- While
- do-While
- foreach

```
string[] names = { "سارا", "رضا", "تیما" };  
foreach (string person in names)  
    Console.WriteLine(person);
```



# CONTROL STATEMENTS



## Conditionals

- If-else
- Switch

## Loops

- For
- While
- do-While
- foreach

## Jump Statements

- Break
- Continue
- Return
- goto

# EXAMPLE



- Prime Numbers

```
static bool IsPrime(int number)
{
    if (number < 2) return false;
    if (number == 2) return true;
    if (number % 2 == 0) return false;

    int limit = (int)Math.Sqrt(number);
    for (int i = 3; i <= limit; i += 2)
    {
        if (number % i == 0) return false;
    }
    return true;
}
```

- Factorial

```
int Factorial(int n)
{
    if (n <= 1) return 1;
    return n * Factorial(n - 1);
}
```

# EXEPTION HANDLING



## Manage by CLR

```
try
{
    int x = 10;
    int y = 0;
    Console.WriteLine(x / y);
}
catch (DivideByZeroException ex)
{
    Console.WriteLine(ex.Message);
}
finally
{
    Console.WriteLine("Finished");
}
```

## Custom Exception

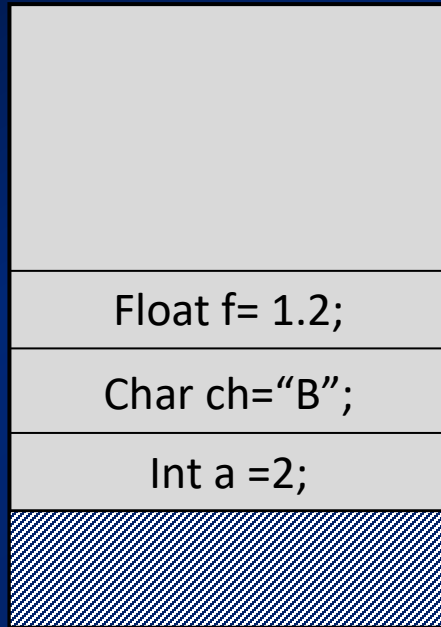
```
class InvalidOrderException : Exception
{
    // references
    public InvalidOrderException(string msg)
        : base(msg)
    {
    }
}
```

# MEMORY MANGEMENT

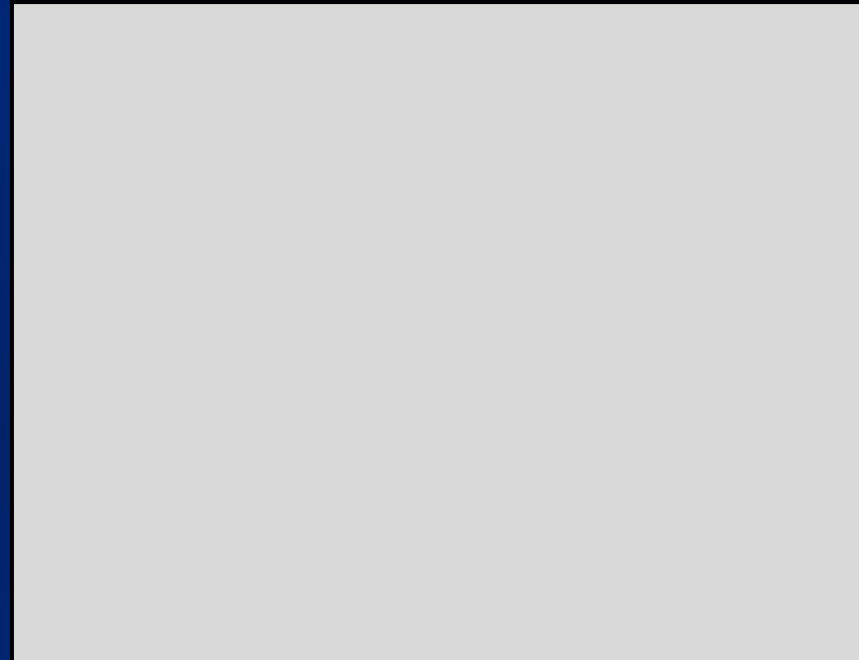
Heap & Stack



- Value Types  
int, float, char
- Delete after  
method ends
- Delete  
Automatically
- Last In First  
Out



Stack  
(fast , temporary)



Heap  
(Big , long term)

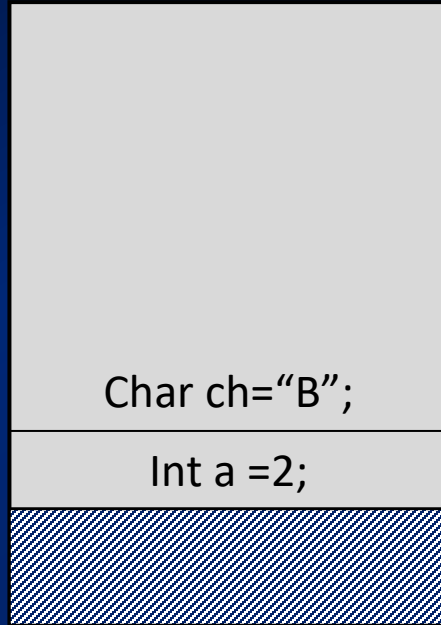
# MEMORY MANGEMENT

Heap & Stack

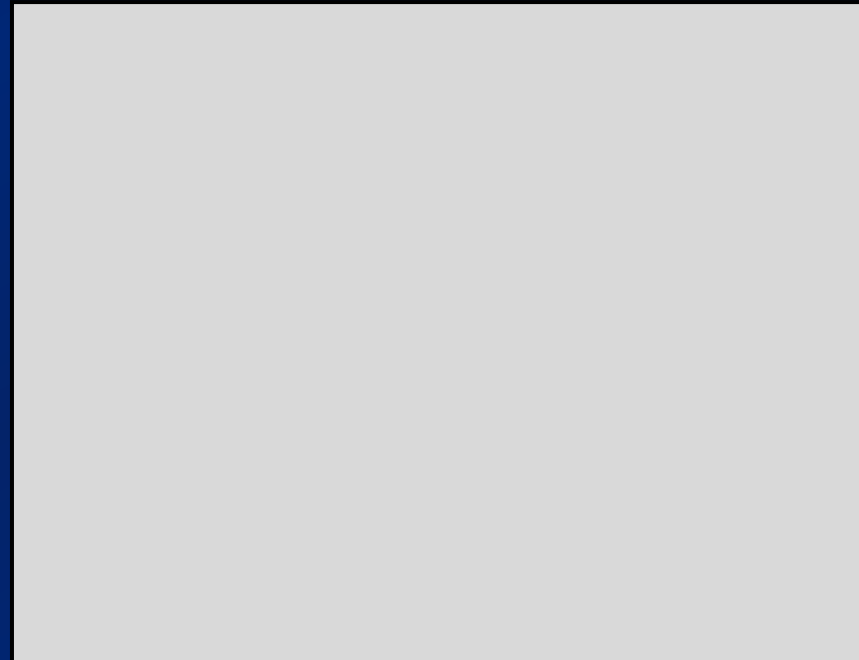


Float f= 1.2;

- Value Types  
int, float, char
- Delete after  
method ends
- Delete  
Automatically
- Last In First  
Out



Stack  
(fast , temporary)



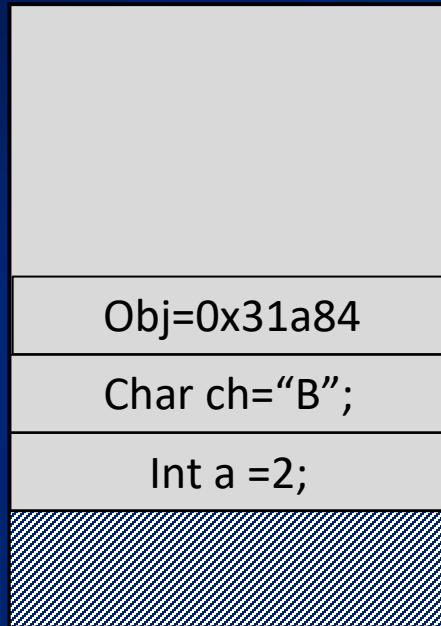
Heap  
(Big , long term)

# MEMORY MANGEMENT

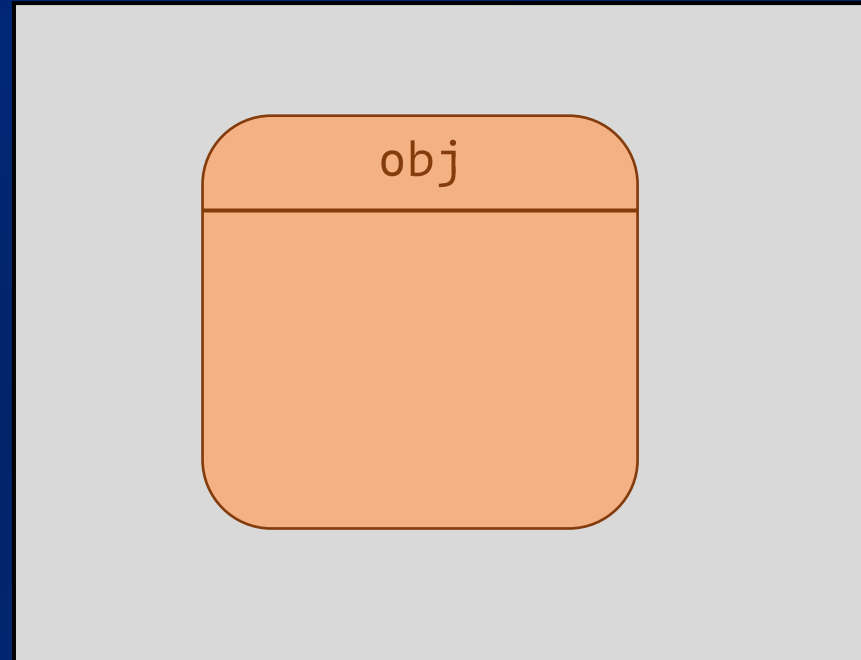
Heap & Stack



- Value Types  
int, float, char
- Delete after  
method ends
- Delete  
Automatically
- Last In First  
Out



Stack  
(fast , temporary)



Heap  
(Big , long term)

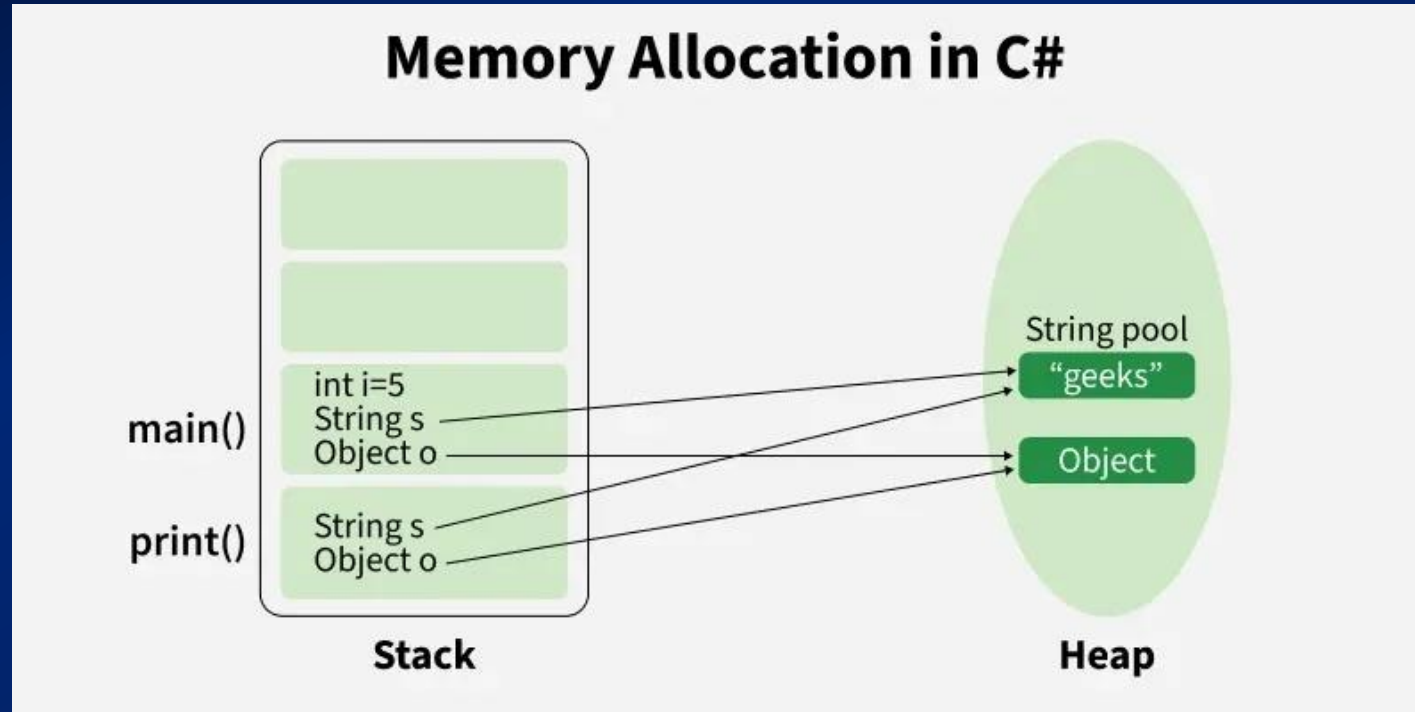
- Refrence Types  
String, Arrays,  
Class
  - Automatic Vs  
Manual Memory  
Deallocation
- Memory Leak  
Crash  
Dangling Pointer

# MEMORY MANGEMENT

Heap & Stack



- Value Types  
int, float, char
- Delete after  
method ends
- Delete  
Automatically
- Last In First  
Out



- Refrence Types  
String, Arrays,  
Class
- Automatic Vs  
Manual Memory  
Deallocation

# MEMORY MANGEMENT

Heap & Stack

Garbage Collector



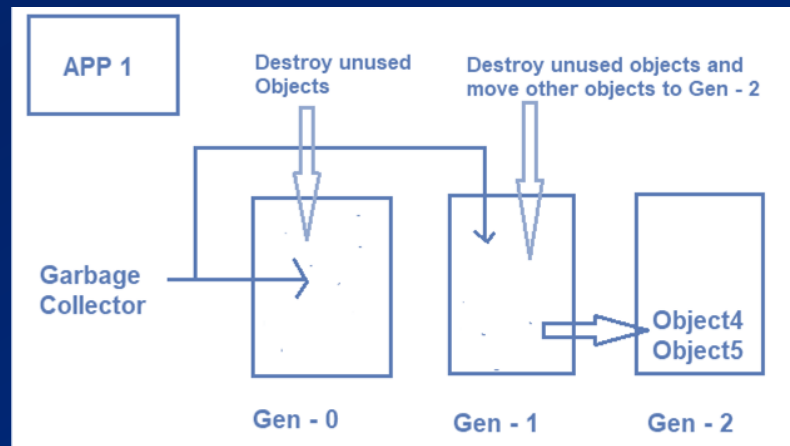
- Mark-Sweep Algorithm:

1. Mark
2. Sweep
3. Compact

| Advantages                  | Disadvantages             |
|-----------------------------|---------------------------|
| Automatic Memory Management | Performance Overhead      |
| Fewer Memory Leaks          | Unpredictable Pause Times |

- Generational

1. Gen 0
2. Gen 1
3. Gen 2





# POINTERS



- Safe Code    NO pointers

- Unsafe Code

- Unsafe
- Value type
- Fixed
- Buffer overflow

```
unsafe
{
    int[] arr = { 1, 2, 3 };

    fixed (int* p = arr)
    {
        Console.WriteLine(*(p + 1)); // 2
    }
}
```

# VERSIONS



2002

C# 1.0

## DELEGATES & EVENTS

```
delegate void EventHandler(object sender, EventArgs e);
class Program
{
    static void Main()
    {
        // ...
    }
}
```

Type Safe  
Coordination Between CLR and  
Runtime Collection

```
public class Observer
{
    public void Notify()
    {
        // ...
    }
}
```

Subscriber  
Subscriber

2005

C# 2.0

## GENERICS

Key advantages:

1. Type safety - Types are determined at compile time, eliminating many runtime errors.
2. No casting required
3. Improved performance
4. Code reusability

```
public class Box<T>
{
    public T Value;
}
Box<int> box = new<>();
box.Value = 10;
```

2007

C# 3.0

## LINQ & LAMBDA EXPRESSION

```
var query = from item in items
             where item > 10
             select item;
// ...
```

Before LINQ

```
var query = items.Where(item => item > 10).Select(item => item);
// ...
```

after LINQ

2012

C# 5.0

## ASYNC & AWAIT

Default: Synchronous

```
async void Async()
{
    int result = await GetAsyncData();
    Console.WriteLine(result);
}
```

2019

C# 8.0

## NULLABLE REFERENCES

```
int? age = 10;
```

```
int? age = null;
if (age.HasValue)
{
    Console.WriteLine(age.Value);
}
```

```
string str = "day";
```

```
string? str = null;
```

2020

C# 9.0

## RECORDS

Value-Based Equality

```
public record Person(string Name, int Age);
Person p1 = new("Ali", 22);
Console.WriteLine(p1.Name);
```

# DELEGATES & EVENTS

```
delegate int Operation(int a, int b);

int Sum(int a, int b)
{
    return a + b;
}

Operation op = Sum;
Console.WriteLine(op(2, 3));
```

Type Safe

Coordination between CLR and  
Garbage Collector

```
public class Alarm
{
    public event Action? Ring;

    public void Start()
    {
        Ring?.Invoke();
    }
}
```

Publisher

Subscriber

# GENERICICS

## Key advantages:

- 1.Type safety – Types are determined at compile time, eliminating many runtime errors.
- 2.No casting required
- 3.Improved performance
- 4.Code reusability

```
public class Box<T>
{
    public T Value;
}

Box<int> number = new();
number.Value = 10;
```

# LINQ & LAMBDA EXPRESSION

```
List<int> numbers = new() {1,2,3,4,5};

foreach(int n in numbers)
{
    if(n > 3)
        Console.WriteLine(n);
}
```

Before LINQ

```
var result = numbers.Where(x => x > 3);
```

after LINQ

# ASYNC & AWAIT

Default : Synchronous

```
async Task Example()  
{  
    int result = await GetNumberAsync();  
  
    Console.WriteLine(result);  
}
```

# NULLABLE REFERENCES

```
int age = 12;
```

```
string str = "Day";
```

```
int? age = null;

if (age.HasValue)
{
    Console.WriteLine(age.Value);
}
```

```
string? str = null;
```

# RECORDS

Value-Based Equality

```
public record Person(string Name, int Age);  
  
Person p1 = new("Ali", 22);  
Console.WriteLine(p1.Name);
```





| Advantages                  | Disadvantages                        |
|-----------------------------|--------------------------------------|
| Automatic Memory Management | Higher Memory Usage than C/C++       |
| Strong Type Safety          | Less Low-Level Control               |
| Rich .NET Ecosystem         | Runtime Dependency                   |
| Modern Language Features    | Garbage Collection Overhead          |
| Cross-Platform Support      | Not Ideal for Hard Real-Time Systems |

C# provides high performance through JIT/AOT compilation and runtime optimizations, although it is generally slightly slower than native C and C++ applications.



# C# Comparison with Other Languages

| Feature           | C         | C++               | Java               | C#                 |
|-------------------|-----------|-------------------|--------------------|--------------------|
| First Release     | 1972      | 1985              | 1995               | 2000               |
| Memory Management | Manual    | Manual            | Garbage Collection | Garbage Collection |
| Execution Model   | Native    | Native            | JVM                | CLR / IL           |
| Performance       | Very High | Very High         | High               | High               |
| Type Safety       | Low       | Medium            | High               | High               |
| Pointer Support   | Full      | Full              | No                 | Limited            |
| Cross Platform    | Limited   | Limited           | Excellent          | Excellent          |
| Async Programming | Manual    | Complex           | Available          | Built-in           |
| Game Development  | Rare      | Industry Standard | Limited            | Strong (Unity)     |



# FUTURE OF C#

- Native AOT compilation for higher performance
- Expansion in cloud computing and microservices
- Stronger AI and machine learning integration
- Growth of cross-platform development with .NET MAUI
- More advanced language features and compiler optimizations
- Continuous improvements to CLR, JIT and Garbage Collector

**THANK YOU**

